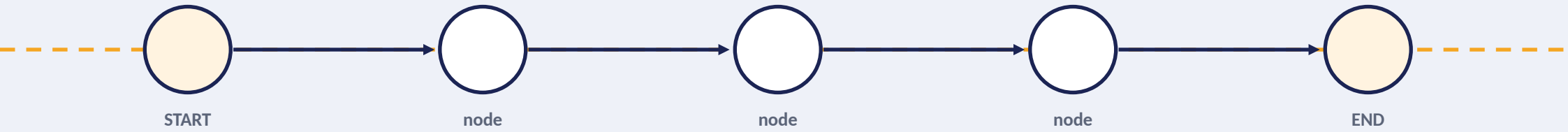
LANGGRAPH · CHAPTER 3 · FLASHCARDS

Reducers and Chat History

Notebooks, not whiteboards

11 code cards on merge rules and a first chatbot



Replace: the silent default

Without a reducer, a returned value overwrites the field's old value

- Right for a field you mean to overwrite, such as the latest score or the current draft.
- Wrong for anything you meant to accumulate: the old items vanish and nothing raises an error.

PYTHON

```python
class StateWithoutReducer(TypedDict):
    count: int
    animals: list[str]


def node_to_update(state: StateWithoutReducer) -> dict:
    return {"count": 1, "animals": ["cat"]}   # brand-new values


app = graph.compile()     # START -> update_node -> END
final_state = app.invoke({"count": 5, "animals": ["lion", "tiger"]})
print(final_state)
# {'count': 1, 'animals': ['cat']}     <- the lion and the tiger are gone
```

A merge rule on the field

Annotated[T, f] attaches a reducer f(current, new) -> combined to one field

- The node is unchanged; only the schema says how its return combines with the old value.
- Try operator.add first: it sums numbers and concatenates lists.

PYTHON

```python
from typing import Annotated
from operator import add


def custom_increment(current: int, new: int) -> int:
    return current + new          # add, don't replace


class StateWithCustomReducer(TypedDict):
    count: Annotated[int, custom_increment]   # our own rule
    animals: Annotated[list[str], add]        # list + list = concatenate


# Same node as before: it still returns {"count": 1, "animals": ["cat"]}
final_state = app.invoke({"count": 5, "animals": ["lion", "tiger"]})
# {'count': 6, 'animals': ['lion', 'tiger', 'cat']}
```

The input goes through the reducer too

invoke() feeds the input to each reducer, starting from the type's empty value (0, [])

- With add the result looks like plain seeding: 0 + 5 is 5 and [] + ["lion"] is ["lion"].
- A rule without that identity shows the call: keep_max(0, -5) stores 0, not -5.

PYTHON

```python
calls = []

def spy(current: int, new: int) -> int:     # a reducer that logs its calls
    calls.append((current, new))
    return current + new

class SpyState(TypedDict):
    count: Annotated[int, spy]

def node(state: SpyState) -> dict:
    return {"count": 1}

app.invoke({"count": 5})        # START -> node -> END
# {'count': 6}
print(calls)
# [(0, 5), (5, 1)]   <- first call is the input, reduced onto int() == 0

# Checked on langgraph 1.2.6 (this snippet is ours, not the book's)
```

Chat is a list of typed messages

The model API is stateless: your code keeps the conversation and sends all of it

- The role is information: system rules, user turns and model replies stay distinguishable.
- Send the whole list and the model can answer "Alex"; send only the last line and it cannot.

PYTHON

```python
from langchain_core.messages import SystemMessage, HumanMessage, AIMessage

conversation = [
    SystemMessage("You are a helpful assistant who answers in one sentence."),
    HumanMessage("My name is Alex. What is RAG?"),
    AIMessage("RAG is Retrieval-Augmented Generation: it fetches "
              "documents and feeds them to the model."),
    HumanMessage("What is my name?"),
]

# SystemMessage  you, the developer: standing instructions
# HumanMessage   the end user
# AIMessage      the model's reply (and, later, its tool requests)
# ToolMessage    a tool's result (Chapter 6)
```

The reducer for chat history

Annotate the messages field with add_messages and each node's new messages are appended

- All four message types inherit from BaseMessage, so one list can hold the whole conversation.
- Without the annotation the final list is just ['Hello from the node!']: the bot forgets.

PYTHON

```python
from langchain_core.messages import BaseMessage, HumanMessage
from langgraph.graph.message import add_messages


class StateWithMessages(TypedDict):
    messages: Annotated[list[BaseMessage], add_messages]


def node_messages_reducer(state: StateWithMessages) -> dict:
    # Return ONE new message; add_messages appends it
    return {"messages": [HumanMessage(content="Hello from the node!")]}


final_state = app.invoke({"messages": [HumanMessage(content="Initial message.")]})
print([m.content for m in final_state["messages"]])
# ['Initial message.', 'Hello from the node!']
```

Append, replace or remove by id

A new id appends, an existing id replaces in place, RemoveMessage(id) deletes

- Replace-by-id lets an agent correct a message; remove-by-id is how long chats get trimmed.
- operator.add can only append: the same id would appear twice in the list.

PYTHON

```python
history = [HumanMessage("My name is Alex.", id="1"),
           AIMessage("Hello Alex.", id="2")]

add_messages(history, [HumanMessage("What is RAG?", id="3")])
# [('1', 'My name is Alex.'), ('2', 'Hello Alex.'), ('3', 'What is RAG?')]

add_messages(history, [AIMessage("Hello, Alex! How can I help?", id="2")])
# [('1', 'My name is Alex.'), ('2', 'Hello, Alex! How can I help?')]

add_messages(history, [RemoveMessage(id="1")])
# [('2', 'Hello Alex.')]

# A reducer is a plain function: call it without a graph to test it
```

Send everything, return the delta

The node sends the whole history to the model and returns only the new reply

- Build the client once at module level: it is shared configuration, not per-run state.
- Return [response], a one-element list, and let add_messages append it to the history.

PYTHON

```python
from langchain_openai import ChatOpenAI
from dotenv import load_dotenv

load_dotenv()                                    # reads OPENAI_API_KEY
llm = ChatOpenAI(model="gpt-4o", temperature=0) # once, not inside the node


class ChatState(TypedDict):
    messages: Annotated[list, add_messages]


def chatbot_node(state: ChatState) -> dict:
    response = llm.invoke(state["messages"])   # send the WHOLE conversation
    return {"messages": [response]}            # return ONLY the new reply
```

Wire the chat node and run one turn

The same START -> node -> END graph, now carrying a growing list of messages

- The final state holds the question and the appended reply, in order.
- History lives only inside one invoke(): run again and the model remembers nothing (Chapter 9).

PYTHON

```python
graph = StateGraph(ChatState)
graph.add_node("chatbot", chatbot_node)
graph.add_edge(START, "chatbot")
graph.add_edge("chatbot", END)
app = graph.compile()

final_state = app.invoke({"messages": [HumanMessage(content="What is RAG?")]})
for message in final_state["messages"]:
    print(f"{message.__class__.__name__}: {message.content}")
# HumanMessage: What is RAG?
# AIMessage: RAG stands for Retrieval-Augmented Generation ...
#            (wording varies; the two-message shape does not)
```

The chat schema, pre-written

MessagesState is a messages field with add_messages; subclass it to add your own fields

- Use the base class for a pure chatbot and a subclass once you need a counter or a user id.
- The subclass keeps messages and its reducer; your extra fields use the default replace rule.

PYTHON

```python
from langgraph.graph import MessagesState, StateGraph

graph = StateGraph(MessagesState)     # same as our hand-written ChatState


class MyGraphState(MessagesState):    # inherits messages + add_messages
    turn_count: int
    user_id: str


def chatbot_node(state: MyGraphState) -> dict:
    response = llm.invoke(state["messages"])
    return {"messages": [response], "turn_count": 1}
# final state: both messages, turn_count 1, user_id unchanged
```

PITFALL · WHAT YOU RETURN

add_messages forgives some returns, not all

A bare string is filed as a HumanMessage; a repeated history is deduplicated by id

- Returning response.content puts the model's reply in the list as if the user had typed it.
- Re-returning the history doubles it only under operator.add; add_messages matches the ids.

PYTHON

```python
def chatbot_node(state: ChatState) -> dict:
    response = llm.invoke(state["messages"])
    return {"messages": [response.content]}     # a str, not the AIMessage
# types: ['HumanMessage', 'HumanMessage']   <- the reply looks like the user's

# Returning the whole history plus the reply:
#     return {"messages": state["messages"] + [response]}
# add_messages: ['What is RAG?', '<reply>']           same ids are replaced
# operator.add: ['What is RAG?', 'What is RAG?', '<reply>']   doubled

# Still return only [response]: it is the clear contract.
# Checked on langgraph 1.2.6 with a fake chat model (no API call)
```

Why the bot forgets

Most chatbot amnesia is one of three lines: the schema, the model call or the run

- A plain messages: list replaces the history; sending only [-1] hides it from the model.
- Each invoke() starts from its input alone; carrying history across runs is checkpointing.

PYTHON

```python
class ChatState(TypedDict):
    messages: list                      # no reducer: each write replaces


def chatbot_node(state: ChatState) -> dict:
    last = state["messages"][-1]
    response = llm.invoke([last])       # the model never sees earlier turns
    return {"messages": [response]}


app.invoke({"messages": [HumanMessage("My name is Alex.")]})
app.invoke({"messages": [HumanMessage("What is my name?")]})
# second run starts from scratch: nothing links it to the first
```